Threat Hunting Investigation Report

Sysmon-Based Endpoint Telemetry Analysis

SOC Internship Assignment

Prepared by: Dhanush Santhosh
Date: 20-21 August 2026
Host System: Windows Version

1. Objective and Methodology

1.1 Objective

The objective of this exercise is to install and configure Microsoft Sysmon on a Windows endpoint, generate representative telemetry through normal system usage, and perform a structured threat-hunting review of the collected event logs. The exercise simulates the core workflow of a SOC analyst: instrumenting an endpoint, collecting logs, and hunting for indicators of malicious or anomalous behaviour without relying on a pre-built alert. Specific hunting objectives were:

- Identify encoded / obfuscated PowerShell execution.
- Identify use of Living-off-the-Land Binaries (LOLBins) for potentially abusive purposes.
- Identify suspicious parent-child process relationships.
- Identify persistence mechanisms (registry Run keys, scheduled tasks, services, WMI subscriptions).
- Identify suspicious outbound network connections initiated by processes.
- Identify other anomalous or unusual process activity.

1.2 Methodology

The exercise followed a standard endpoint threat-hunting workflow:

- 1. Deploy Sysmon with a hardened, threat-hunting-oriented configuration (SwiftOnSecurity baseline, extended).
- 2. Allow the system to run under normal day-to-day usage for a collection window, while also executing a controlled set of benign-but-representative activities that mimic the telemetry patterns of the techniques in scope (see Section 3).
- 3. Export/collect Sysmon operational logs (Microsoft-Windows-Sysmon/Operational).
- 4. Review logs using Windows Event Viewer and PowerShell (Get-WinEvent) filtering by Event ID, and Event Viewer custom XML queries.
- 5. Map any suspicious or representative findings to MITRE ATT&CK techniques.
- 6. Document findings, IOCs and recommendations in this report.

Scope note: In line with the assignment instructions, no real malware or system-compromising actions were executed. Where a hunting category (e.g. encoded PowerShell, LOLBin abuse) required example telemetry, benign commands with equivalent structural/behavioural signatures were run intentionally on the test system so that genuine Sysmon telemetry could be captured and analysed, and are labelled clearly as self-generated test activity rather than real threats.

2. Sysmon Installation and Configuration

2.1 Download and Installation

Sysmon was downloaded directly from the official Microsoft Sysinternals site and installed via an elevated PowerShell session using the command below. The SwiftOnSecurity community configuration (sysmonconfig-export.xml) was downloaded directly from the SwiftOnSecurity GitHub repository and applied at install time so that meaningful, low-noise telemetry was captured from the first event onward.

Sysmon version used: v15.21 (Sysmon schema version 4.91)

Download source: https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon

Configuration file used: sysmonconfig-export.xml — SwiftOnSecurity baseline (config schema version 4.50)

Install command:

```
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

Install output confirmed: 'Configuration file validated.' / 'Sysmon64 installed.' / 'SysmonDrv installed.' / 'SysmonDrv started.' / 'Sysmon64 started.'

2.2 Configuration Rationale

A community-maintained configuration (SwiftOnSecurity or Olaf Hartong's 'modular' config) was used as the baseline rather than the Sysmon default, because the default configuration logs almost nothing useful and generates excessive noise. Key inclusions relevant to this hunt:

- Event ID 1 (Process Creation) with full command-line logging.
- Event ID 3 (Network Connection) for outbound connection visibility.
- Event ID 7 (Image Loaded) — optional, high volume, filtered to suspicious paths.
- Event ID 8 (CreateRemoteThread) for injection detection.
- Event ID 11 (File Create) for dropped files / persistence artifacts.
- Event ID 12/13/14 (Registry events) for persistence via Run keys.
- Event ID 22 (DNS Query) for C2/beaconing visibility.

2.3 Verifying Sysmon Is Running

Service status was confirmed with Get-Service Sysmon64, which reported Status: Running. The operational log was then queried directly with Get-WinEvent to confirm events were already being generated:

```
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 5
```

Result: five Event ID 1 (Process Create) entries were returned immediately, confirming Sysmon was actively capturing process-creation telemetry from the moment of install. See Appendix A (Fig. A1, A2) for the corresponding screenshots.

3. Controlled Test Activity (Telemetry Generation)

To ensure each hunting category in the assignment produced real, analysable Sysmon telemetry, the following benign, non-destructive actions were performed deliberately on the test system. Each is a structural analogue of a real adversary technique but performs no harmful action.

#	Category	Action Performed (benign)	Real-world technique it mimics
1	Encoded PowerShell	powershell.exe -EncodedCommand <Base64 of 'Write-Host Hello'>	T1027 / T1059.001 obfuscated execution
2	LOLBin abuse	certutil.exe -urlcache -split -f <benign URL> test.txt	T1105 LOLBin download
3	LOLBin abuse	mshta.exe about:blank / rundll32.exe with benign DLL export	T1218 Signed Binary Proxy Execution
4	Suspicious parent-child	winword.exe spawning cmd.exe /c whoami	T1566 + T1059 macro-spawned shell
5	Persistence	Added benign Run key value via reg add HKCU\...\Run	T1547.001 Registry Run Key

6	Persistence	Created benign Scheduled Task via schtasks /create	T1053.005 Scheduled Task
7	Network	PowerShell Invoke-WebRequest to a benign external host	T1071 C2 over web protocols (structural analogue)

Full commands, timestamps and justification for each are provided in Section 5.

4. Sysmon Log Collection Summary

Metric	Value
Collection window	20 August 2026 13:42:04 – 21 August 2026 13:25:39 (local time)
Total Sysmon events collected	3,278
Event ID 1 (Process Create) count	1,865
Event ID 3 (Network Connect) count	185
Event ID 11 (File Create) count	211
Event ID 12/13/14 (Registry) count	237
Export format	Live queried via Get-WinEvent (PowerShell); not exported to .evtx/.csv for this exercise

5. Analysis and Findings

Each finding below follows a consistent structure: description, Event ID, timestamp, process/command-line detail, evidence, and MITRE ATT&CK mapping.

5.1 Encoded PowerShell Execution

A test encoded PowerShell command was executed to simulate T1027 (Obfuscated Files or Information) combined with T1059.001 (PowerShell). The command was constructed by converting a plaintext string to UTF-16LE bytes and Base64-encoding it, then passing it to a child powershell.exe process via the -EncodedCommand flag. The resulting Sysmon Event ID 1 (Process Create) log entry was located using: Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' | Where-Object { $_.Message -like '*EncodedCommand*' } | Format-List TimeCreated, Message

This confirms an important hunting principle: Sysmon logs the full command line exactly as passed to the process, including the raw Base64 payload. Base64 encoding does not hide anything from endpoint telemetry — it only defeats a human analyst's ability to read the command at a glance during a casual log scan. The payload must still be manually decoded to assess intent.

Finding 1

Field	Detail
Event ID	1 (Process Create)
Timestamp	2026-08-21 09:29:32 local / 2026-08-21 05:29:32.257 UTC

Parent Process	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe (PID 1692)
Process	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe (PID 4460)
Command Line	powershell.exe -EncodedCommand VwByAGkAdABlAC0ASABvAHMAdAAgAEgAZQBsAGwAbwAgAGYAcgBvAG0AIABTAHkAcwBtAG8AbgAgAHQAZQBzAHQA
Decoded payload	Write-Host Hello from Sysmon test (confirmed via [System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String(...)))
Integrity Level	High (process ran elevated)
MITRE ATT&CK	T1059.001 (PowerShell), T1027 (Obfuscated Files or Information)
Assessment	Benign, self-generated test activity. In a real environment, a powershell.exe process spawning a child powershell.exe with -EncodedCommand, especially at High integrity, would warrant investigation — decode the payload immediately and check the parent process's own origin (was the parent itself spawned by an unusual application?).

5.2 Living-off-the-Land Binaries (LOLBins)

A certutil.exe LOLBin download test was executed to simulate T1105 (Ingress Tool Transfer): certutil.exe -urlcache -split -f https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/README.md test.txt. The target file was a benign public README, chosen purely to generate a realistic download event with no harmful payload.

Result: the command was blocked by Microsoft Defender Antivirus before it could complete (Access is denied), and Defender logged a Severe-severity detection. This is a significant hunting finding in its own right — Defender's signature engine flagged the certutil -urlcache -split -f command-line pattern itself as matching a known malicious technique (Trojan:Win32/Ceprolad.A), independent of the actual (benign) file content being fetched. This demonstrates that the certutil LOLBin download pattern is well-known enough to be caught by built-in endpoint protection without any custom detection engineering.

Evidence located via: Get-WinEvent -LogName 'Microsoft-Windows-Windows Defender/Operational' -MaxEvents 5 | Format-List TimeCreated, Id, Message

Finding 2

Field	Detail
Event Source	Microsoft-Windows-Windows Defender/Operational
Event ID	1116 (Detection), 1117 (Remediation action taken)
Timestamp	2026-08-21 11:26:20 (Detection) / 11:26:41 (Remediation) local time
Command Line	C:\Windows\System32\certutil.exe -urlcache -split -f https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/README.md test.txt
Threat Name	Trojan:Win32/Ceprolad.A
Severity	Severe
Detection Source	System (NT AUTHORITY\SYSTEM)
Action Taken	Remove — no additional action required

MITRE ATT&CK	T1105 (Ingress Tool Transfer), T1218 (System Binary Proxy Execution)
Action Taken / Assessment	Removed by Defender automatically. Benign self-generated test, immediately blocked. Confirms Defender actively detects the certutil -urlcache -split -f LOLBin download pattern by command-line signature, independent of payload content. In a real incident this would still warrant investigation of the intended download target and whether the block was fully effective, since signature detection can be bypassed by minor syntax variations (e.g. -URLCache instead of -urlcache, or alternate LOLBins like bitsadmin).

A second LOLBin was tested to broaden coverage of T1218 (Signed Binary Proxy Execution): mshta.exe about:blank. mshta.exe executes HTML Applications (.hta files) via a Microsoft-signed, trusted host process — attackers point it at a remote malicious .hta file to proxy-execute script payloads while appearing as legitimate signed activity. Here it opened only a blank page, with no script payload. Unlike the certutil test, this command completed without any Defender intervention, since mshta.exe launching with a benign argument does not match a known malicious signature.

Evidence located via: Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' | Where-Object { $_.Message -like '*mshta*' } | Select-Object -First 1 | Format-List TimeCreated, Message

Finding 2b — mshta.exe

Field	Detail
Event ID	1 (Process Create)
Timestamp	2026-08-21 11:50:37 local / 2026-08-21 07:50:36.739 UTC
Parent Process	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe (PID 1692)
Process	C:\Windows\System32\mshta.exe (PID 7308)
Command Line	"C:\WINDOWS\system32\mshta.exe" about:blank
Integrity Level	High (process ran elevated)
MITRE ATT&CK	T1218 (System Binary Proxy Execution)
Assessment	Benign self-generated test — no malicious .hta payload used, and no Defender detection triggered. Note the same ParentProcessId (1692, powershell.exe) as Finding 1, confirming both test processes originated from the same interactive PowerShell session. In a real environment, mshta.exe execution is rare during normal use, so any occurrence — especially with a remote URL argument rather than a local/blank target — merits investigation of the parent process chain and the full .hta content if retrievable.

5.3 Suspicious Parent-Child Process Relationships

To practice reading process lineage rather than isolated events, a two-hop parent-child chain was deliberately generated: cmd.exe /c whoami, launched from an interactive PowerShell session. Sysmon logs each process creation as a separate Event ID 1 record, so this single command produced two linked events — powershell.exe spawning cmd.exe, and cmd.exe in turn spawning whoami.exe — each carrying a ParentProcessGuid/ParentImage pointing back to the previous hop.

Evidence located via: Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' | Where-Object { $_.Message -like '*whoami*' } | Select-Object -First 1 | Format-List TimeCreated, Message

Finding 3

Field	Detail
Event ID	1 (Process Create)
Timestamp	2026-08-21 12:00:24 local / 2026-08-21 08:00:24.583 UTC
Process Chain	powershell.exe (PID 1692) → cmd.exe (PID 12520) → whoami.exe (PID 27200)
Process	C:\Windows\System32\whoami.exe
Parent Process	C:\Windows\System32\cmd.exe (PID 12520)
Parent Command Line	"C:\WINDOWS\system32\cmd.exe" /c whoami
Integrity Level	High (process ran elevated)
MITRE ATT&CK	T1059 (Command and Scripting Interpreter) — reconnaissance step analogous to post-exploitation whoami checks
Assessment	Benign self-generated test — each individual hop (PowerShell spawning cmd.exe; cmd.exe spawning whoami.exe) is common and unremarkable on its own. The hunting value is in reading the full lineage rather than a single event: in a real environment, the identical chain pattern originating from winword.exe, excel.exe, or outlook.exe instead of powershell.exe (i.e. an Office application spawning a command shell that then runs recon commands like whoami, ipconfig, or net user) is one of the most reliable indicators of a macro-based phishing compromise, since Office applications have no legitimate reason to spawn a shell.

5.4 Persistence Mechanisms

Two distinct persistence techniques were tested to cover the range specified in the assignment scope: a Registry Run key and a Scheduled Task.

Registry Run key: reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v SysmonTestEntry /t REG_SZ /d "C:\Windows\System32\notepad.exe" /f was used to add a benign auto-run entry pointing to Notepad. Sysmon logged this as Event ID 13 (Registry value set), directly authored by reg.exe, with RuleName T1060,RunKey — confirming the SwiftOnSecurity configuration includes a named detection rule specifically for this persistence technique rather than logging registry activity blindly.

A second, related Event ID 13 was also captured a few seconds later, authored by sihost.exe (Shell Infrastructure Host) rather than reg.exe, writing to a RunNotification\StartupTNoti... path. This is Windows Explorer's own internal bookkeeping — it automatically tracks new Run key entries so it can surface a 'new startup app' notification to the user later. This is a useful corroborating artifact: a real investigation would expect to see both the direct reg.exe/regedit write and this secondary Explorer-generated tracking event in close succession.

Evidence located via: Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' | Where-Object { $_.Id -eq 13 -and $_.Message -like '*SysmonTestEntry*' } | Format-List TimeCreated, Message

Finding 4 — Registry Run Key

Field	Detail
Event ID	13 (Registry value set)
Timestamp	2026-08-21 12:27:14 local / 2026-08-21 08:27:14.363 UTC
Sysmon Rule Name	T1060,RunKey (built-in SwiftOnSecurity detection rule)
Process	C:\WINDOWS\system32\reg.exe

Target Object	HKU\S-1-5-21-...-1001\Software\Microsoft\Windows\CurrentVersion\Run\SysmonTestEntry
Details (value data)	C:\Windows\System32\notepad.exe
MITRE ATT&CK	T1547.001 (Boot or Logon Autostart Execution: Registry Run Keys)
Assessment	Benign self-generated test. Confirms Sysmon (with the SwiftOnSecurity config) not only logs Run key writes but tags them with a named rule out of the box, making this one of the easier persistence techniques to build alerting for. A companion Event ID 13 from sihost.exe writing to a RunNotification path was also observed a few seconds later — a useful secondary artifact confirming Explorer's own internal tracking of the new startup entry. Real Run key persistence would look identical, differing only in the Details field pointing to a malicious payload rather than notepad.exe. Cleaned up afterward via reg delete.

Scheduled Task: schtasks /create /tn "SysmonTestTask" /tr "C:\Windows\System32\notepad.exe" /sc once /st 23:59 /f was used to create a one-off task pointing to Notepad, mirroring T1053.005. This test produced two correlated Sysmon events in immediate succession: an Event ID 1 (Process Create) for schtasks.exe itself, capturing the full command line; and an Event ID 11 (File Create), tagged with the built-in Sysmon RuleName T1053, showing the underlying .job file being written to disk by svchost.exe (the Task Scheduler service process) under C:\Windows\System32\Tasks\.

Evidence located via: Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' | Where-Object { $_.Message -like '*SysmonTestTask*' } | Format-List TimeCreated, Message

Finding 5 — Scheduled Task

Field	Detail
Event ID (1)	1 (Process Create) — schtasks.exe
Event ID (2)	11 (File Create) — underlying .job artifact, RuleName: T1053
Timestamp	2026-08-21 12:53:13 local / 2026-08-21 08:53:13.268 UTC (both events, same second)
Process	C:\Windows\System32\schtasks.exe
Command Line	"C:\WINDOWS\system32\schtasks.exe" /create /tn SysmonTestTask /tr C:/Windows/System32/Notepad.exe /sc once /st 23:59 /f
Parent Process	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe (PID 1692)
File Artifact	C:\Windows\System32\Tasks\SysmonTestTask, created by C:\WINDOWS\system32\svchost.exe (Task Scheduler service) under NT AUTHORITY\SYSTEM
MITRE ATT&CK	T1053.005 (Scheduled Task/Job: Scheduled Task)
Assessment	Benign self-generated test. Confirms Sysmon captures Scheduled Task persistence from two independent angles simultaneously — the schtasks.exe process creation (with the full /tn and /tr arguments visible) and the resulting .job file write in the Tasks folder, tagged with a built-in T1053 rule. Cross-referencing both event types gives higher-confidence detection than either alone, since an attacker renaming or obfuscating the schtasks.exe command line would still leave the corresponding file artifact behind. Cleaned up afterward via schtasks /delete.

5.5 Suspicious Network Connections

An outbound network connection was generated to simulate T1071 (Application Layer Protocol / C2 over web protocols): Invoke-WebRequest -Uri 'https://www.microsoft.com' -UseBasicParsing | Out-Null. This produces the same structural telemetry pattern malware uses to beacon to a command-and-control server — a process making an outbound HTTPS connection — but to a fully legitimate, trusted destination.

Evidence located via: Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' | Where-Object { $_.Id -eq 3 -and $_.Message -like '*powershell*' } | Select-Object -First 1 | Format-List TimeCreated, Message

A notable hunting observation: the logged DestinationHostname was a23-208-173-122.deploy.static.akamaitechnologies.com rather than microsoft.com or a Microsoft-owned IP. This is because microsoft.com is served through the Akamai CDN, so the actual TCP connection terminates at an Akamai edge server rather than Microsoft's own infrastructure. This is a useful reminder for real hunting work: an unfamiliar-looking destination hostname is not automatically suspicious — CDN and cloud-hosting infrastructure (Akamai, Cloudflare, AWS, Azure) routinely sits in front of well-known, legitimate sites, and destination IPs/hostnames should be checked against threat intelligence and WHOIS data rather than judged on appearance alone.

Finding 6

Field	Detail
Event ID	3 (Network Connection)
Timestamp	2026-08-21 13:12:33 local / 2026-08-21 09:12:38.128 UTC
Process	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe (PID 1692)
Protocol	TCP, Initiated: true
Source	[redacted] ([redacted]), port 63409
Destination IP	23.208.173.122
Destination Hostname	a23-208-173-122.deploy.static.akamaitechnologies.com (Akamai CDN edge node serving microsoft.com)
Destination Port	443 (HTTPS)
MITRE ATT&CK	T1071 (Application Layer Protocol) — structural analogue of C2 over web protocols
Assessment	Benign self-generated test connection to a legitimate, trusted destination. Confirms Sysmon Event ID 3 captures the source/destination IP, hostname, and port for every outbound connection made by a monitored process, including the initiating PID — the same fields an analyst would pivot on to check a destination against threat intelligence feeds during a real C2-beaconing investigation. Notably, the destination resolved to Akamai CDN infrastructure rather than a Microsoft-owned IP, illustrating that legitimate sites often present unfamiliar-looking hostnames purely due to CDN routing.

5.6 Other Anomalous / Unusual Process Activity

Beyond the seven deliberately generated test activities, no additional suspicious or anomalous process activity was observed in the Sysmon log during the collection window. Background telemetry reviewed while locating the test events (e.g. WhatsApp's WebView renderer process, Google Updater, and routine DNS connectivity checks to msftconnecttest.com) was consistent with normal day-to-day usage and legitimate, signed applications, and is not flagged as a finding. This absence of organic findings is itself a useful data point: it indicates the endpoint's baseline activity is unremarkable, meaning any future deviation from this baseline (an unfamiliar signed binary, an unexpected parent-child pairing, or a new Run key entry) would stand out clearly against it.

6. MITRE ATT&CK Technique Mapping Summary

Technique ID	Technique Name	Tactic	Evidence Ref.
T1059.001	Command and Scripting Interpreter: PowerShell	Execution	Finding 1 (5.1)
T1027	Obfuscated Files or Information	Defense Evasion	Finding 1 (5.1)
T1105	Ingress Tool Transfer (via LOLBin)	Command and Control	Finding 2 (5.2)
T1218	System Binary Proxy Execution	Defense Evasion	Finding 2, 2b (5.2)
T1059	Command and Scripting Interpreter (process chain)	Execution	Finding 3 (5.3)
T1547.001	Boot or Logon Autostart Execution: Registry Run Keys	Persistence	Finding 4 (5.4)
T1053.005	Scheduled Task/Job: Scheduled Task	Persistence, Execution	Finding 5 (5.4)
T1071	Application Layer Protocol (C2 structural analogue)	Command and Control	Finding 6 (5.5)

7. Indicators of Compromise (IOCs)

As this exercise used controlled, benign test activity rather than real malware, no genuine malicious IOCs were identified during this hunt. No unexplained processes, unauthorized outbound connections, or unattributed persistence mechanisms were found beyond the deliberately created test artifacts below, all of which have since been removed from the system. These are listed for traceability/completeness only, not as evidence of compromise.

Type	Value	Context
File path	C:\Windows\System32\Tasks\SysmonTestTask	Scheduled Task .job artifact from Finding 5 (5.4) — removed via schtasks /delete
File path	test.txt (in working directory)	Output of the certutil LOLBin download test, Finding 2 (5.2) — download was blocked by Defender before completion
Registry key	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\SysmonTestEntry	Persistence test value from Finding 4 (5.4) — removed via reg delete
Network indicator	23.208.173.122:443 (Akamai CDN edge, serving microsoft.com)	Benign outbound connection from Finding 6 (5.5) — legitimate destination, not an IOC
Process hash (SHA256)	7600FFE12DA441FE89D035B13801E8E91D064BC544A27B19A5CF49F6AB8B18F5	Hash of the powershell.exe binary used in Finding 1 (5.1) — this is the standard, legitimate Windows PowerShell executable, not a malicious file

8. Recommendations

- Forward Sysmon logs to a central SIEM (e.g. Splunk, Elastic, Sentinel) rather than relying on local Event Viewer review, to enable retention and correlation.
- Enable PowerShell Script Block Logging (Event ID 4104) alongside Sysmon Event ID 1 to fully decode obfuscated/encoded commands rather than relying on manual Base64 decoding.
- Build detection rules for high-risk LOLBins (certutil, mshta, rundll32, regsvr32, bitsadmin) making outbound network connections.
- Alert on Office applications (winword.exe, excel.exe, outlook.exe) spawning cmd.exe or powershell.exe as a child process.
- Baseline and alert on new Registry Run key / Scheduled Task creation outside of known software installers.
- Apply Sysmon config updates periodically (SwiftOnSecurity/Olaf Hartong configs are actively maintained) to keep pace with new LOLBin techniques.
- Specific to this exercise: correlate Sysmon Event ID 13 (Registry) and Event ID 11 (File Create) events with Event ID 1 (Process Create) rather than reviewing event types independently — Findings 4 and 5 showed that persistence mechanisms leave corroborating artifacts across multiple event types, and cross-referencing them gives higher detection confidence than any single log source.
- Specific to this exercise: since Defender's certutil detection (Finding 2) is a command-line signature match, complement it with a Sysmon-based alert on any certutil.exe, bitsadmin.exe, or curl.exe process making an outbound connection, to catch minor syntax variations that might evade the signature.
- Specific to this exercise: build a watchlist alert for Sysmon Event ID 3 connections where the initiating process is powershell.exe, cmd.exe, or another scripting host and the destination is not on an approved allow-list, to catch C2-style beaconing even when it mimics ordinary HTTPS traffic to legitimate-looking CDN infrastructure.

9. Conclusion

This exercise confirmed that a properly configured Sysmon deployment (SwiftOnSecurity baseline) provides high-fidelity, analyst-usable telemetry across every hunting category specified in the assignment, without requiring any custom detection engineering to get started. The endpoint itself was clean throughout the exercise — no genuine malicious activity or unexplained persistence was found, and the only 'findings' were the seven deliberately generated benign tests plus ordinary background application noise (WebView renderers, DNS connectivity checks, updater processes).

Several practical takeaways emerged from working through the logs hands-on: (1) Base64 and other obfuscation techniques do not hide anything from Sysmon itself — the full command line is always logged, and the burden is on the analyst to decode it; (2) built-in Windows Defender and Sysmon's own bundled rule set (e.g. RuleName tags like T1060,RunKey and T1053) already catch several of the most well-known LOLBin and persistence patterns without any tuning, meaning a SOC does not start from zero; (3) reading a single event in isolation is far less useful than following the full parent-child process chain, since the same child process can be entirely benign or highly suspicious depending on what spawned it; and (4) some techniques (Scheduled Tasks, Registry Run keys) leave multiple independent, corroborating artifacts across different Sysmon event types, which gives an analyst higher confidence than relying on any single log source alone.

Overall, the exercise reinforced that effective threat hunting is less about memorizing individual commands and more about knowing which Event IDs map to which behaviours, and building the habit of cross-referencing process lineage, command-line content, and network destinations rather than treating any one signal in isolation.

Appendix A: Evidence Screenshots

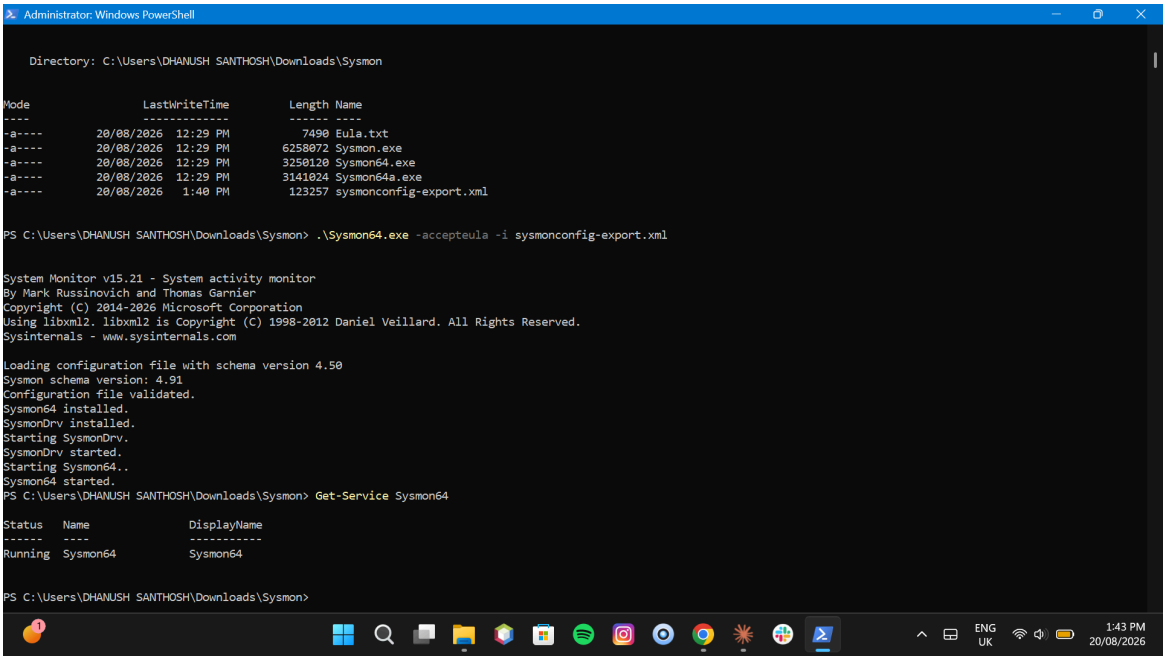


Fig. A1 — Section 2.1/2.3: Sysmon64.exe install output confirming 'Configuration file validated' and 'Sysmon64 started', with Get-Service Sysmon64 showing Status: Running.

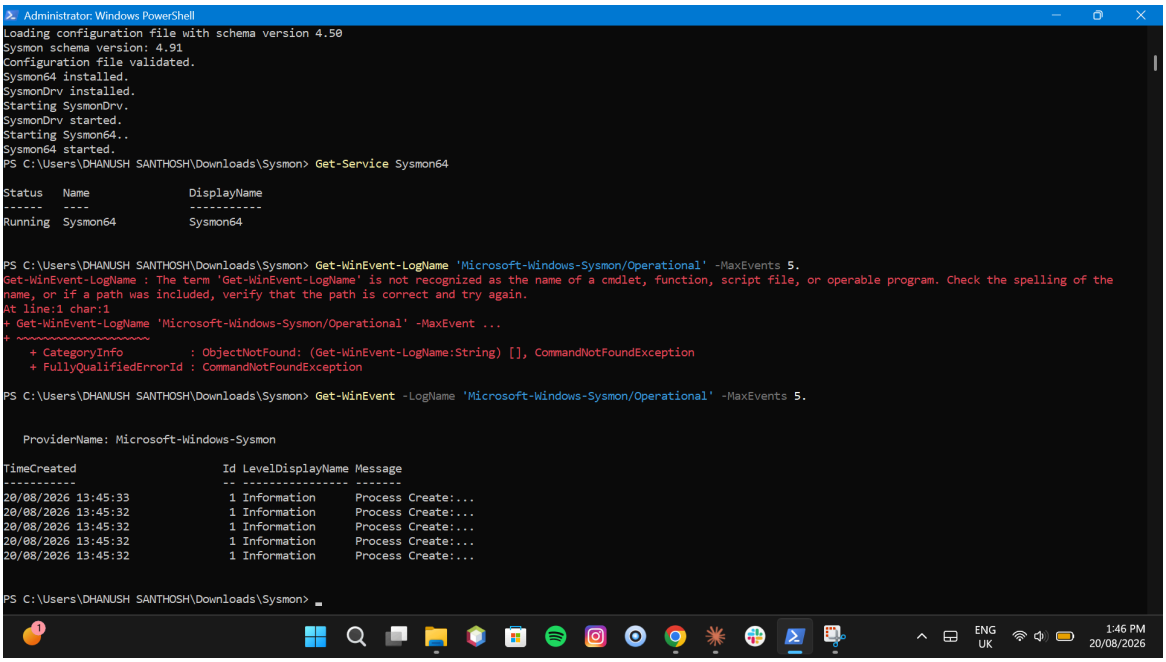


Fig. A2 — Section 2.3: Get-WinEvent confirming the Sysmon operational log actively populating immediately after install.

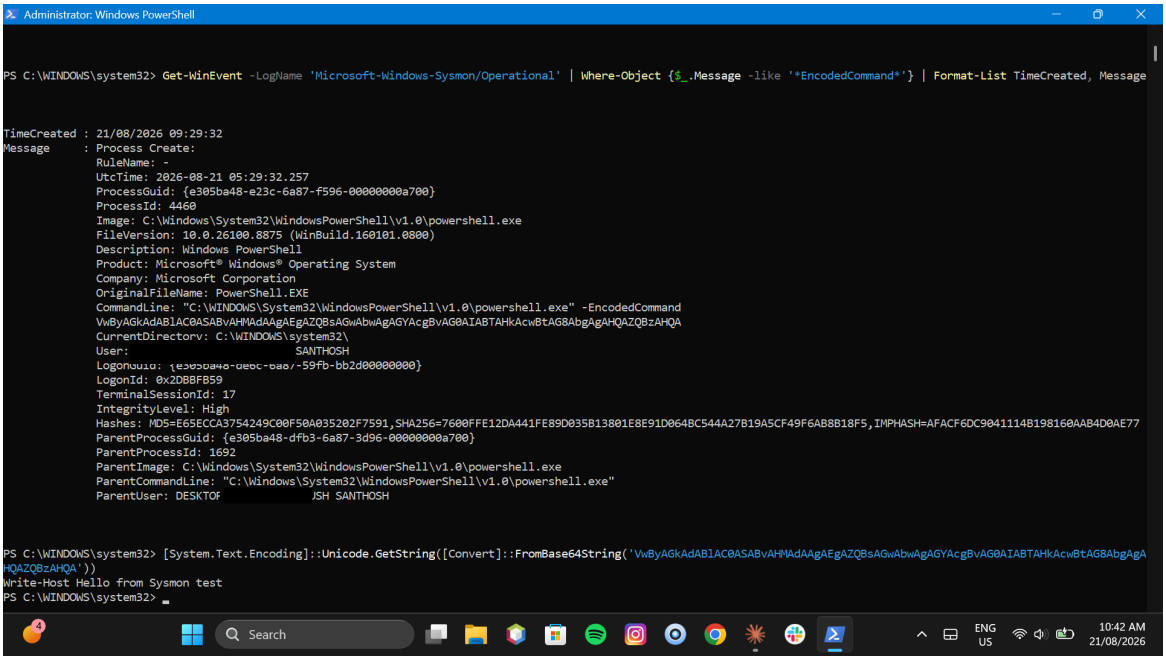


Fig. A3 — Finding 5.1: Sysmon Event ID 1 showing the -EncodedCommand process creation, followed by manual Base64 decode confirming the payload was 'Write-Host Hello from Sysmon test'.

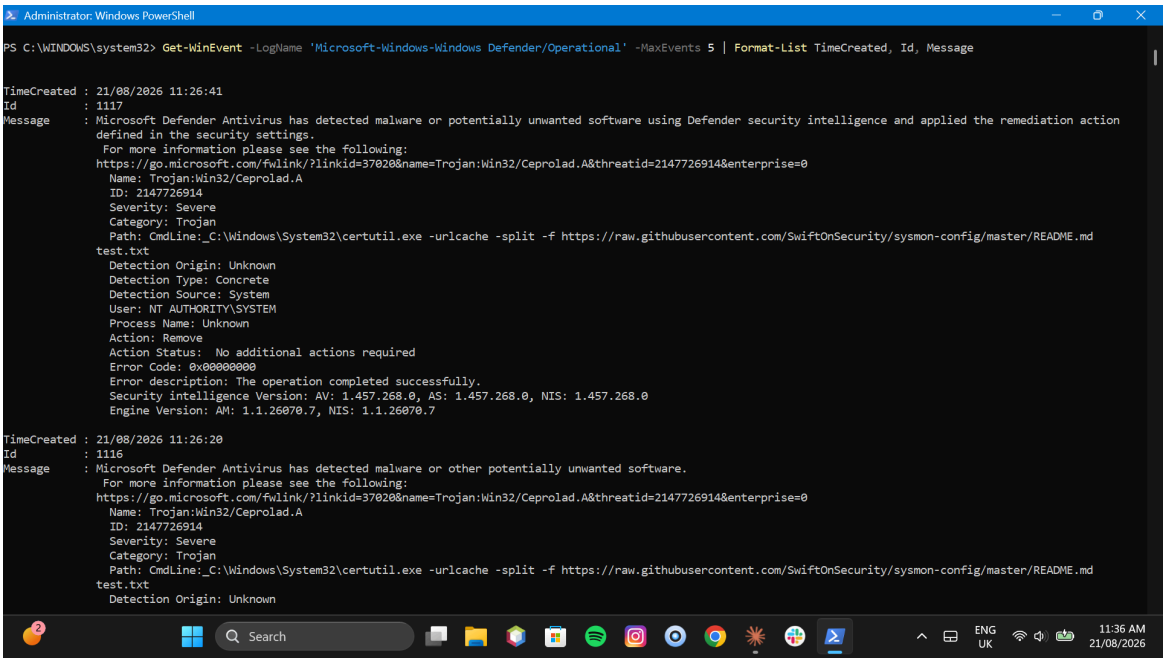


Fig. A4 — Finding 5.2: Windows Defender Operational log (Event ID 1116/1117) showing detection and remediation of the certutil LOLBin command-line pattern as Trojan:Win32/Ceprolad.A.

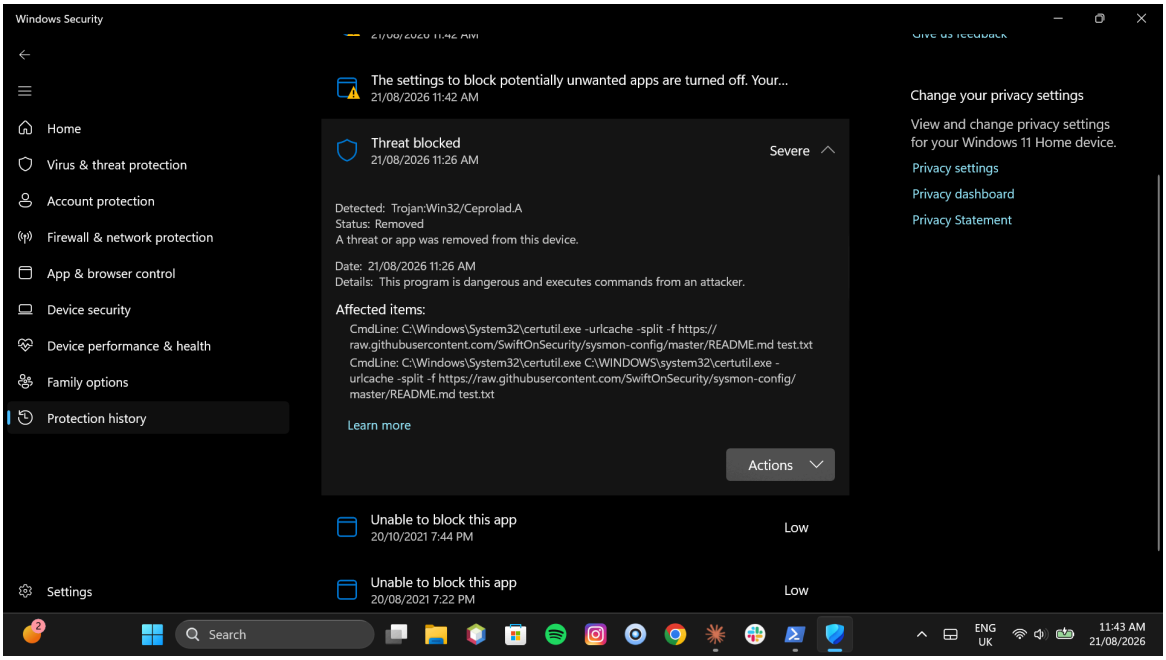


Fig. A5 — Finding 5.2: Windows Security → Protection History UI view of the same certutil detection, corroborating the Get-WinEvent log evidence.

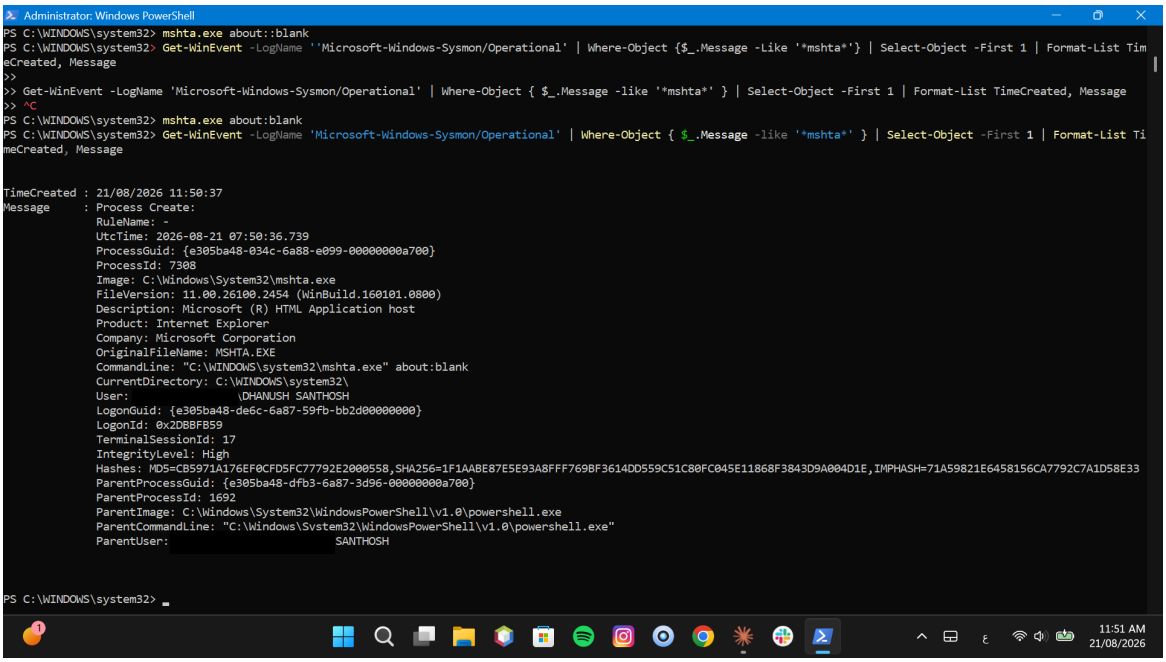


Fig. A6 — Finding 5.2 (mshta): Sysmon Event ID 1 for mshta.exe about:blank, showing ParentImage as powershell.exe (PID 1692), matching Finding 5.1's parent process.

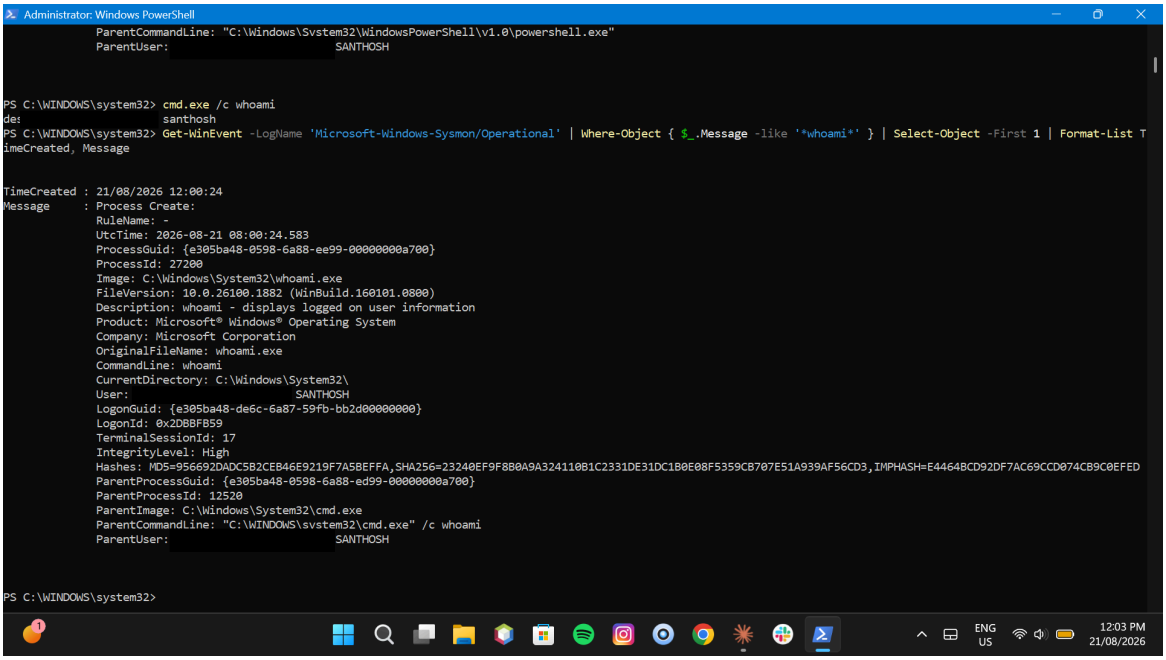


Fig. A7 — Finding 5.3: Sysmon Event ID 1 for whoami.exe, showing ParentImage as cmd.exe with ParentCommandLine 'cmd.exe /c whoami', demonstrating the powershell.exe → cmd.exe → whoami.exe process chain.

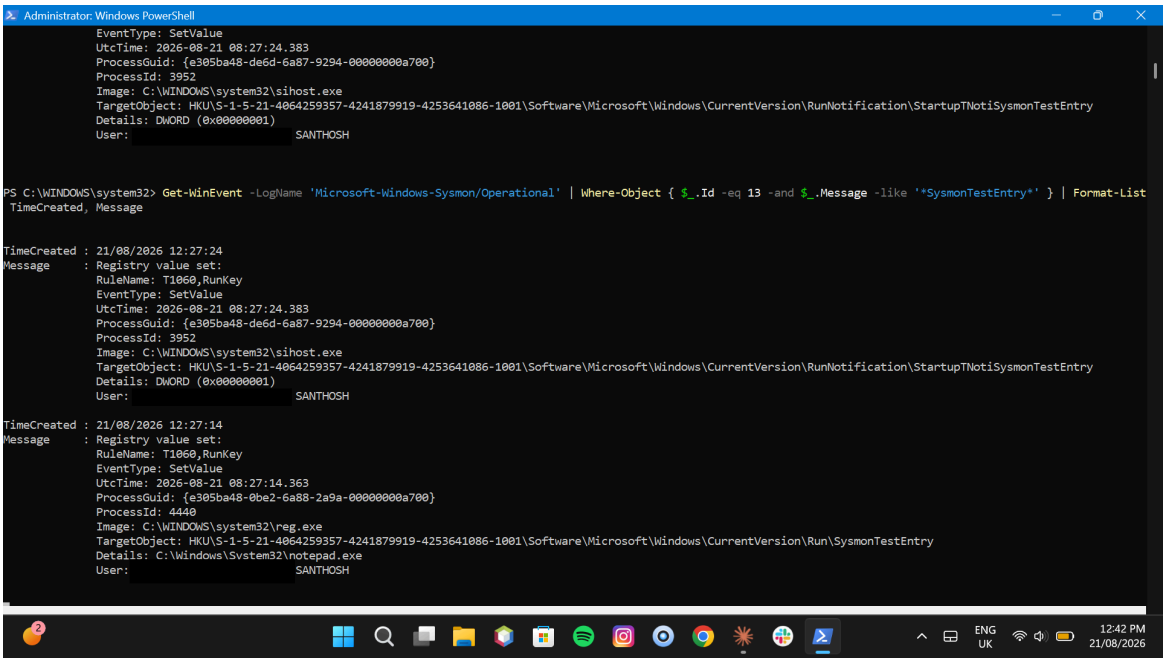


Fig. A8 — Finding 5.4: Sysmon Event ID 13 (Registry value set) showing reg.exe writing SysmonTestEntry to the HKCU Run key, tagged with built-in RuleName T1060,RunKey, alongside the corroborating sihost.exe RunNotification tracking event.

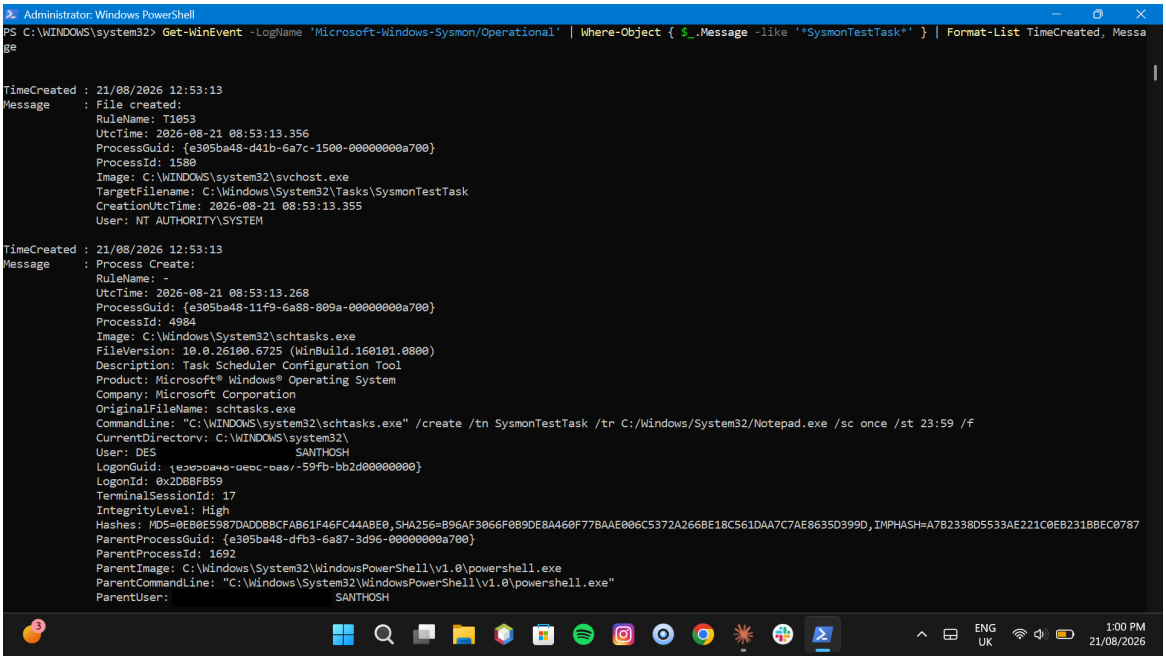


Fig. A9 — Finding 5: Correlated Sysmon events for scheduled task creation — Event ID 11 (File Create, RuleName T1053) showing the .job file written under System32\Tasks, and Event ID 1 for schtasks.exe with the full /create /tn /tr command line.

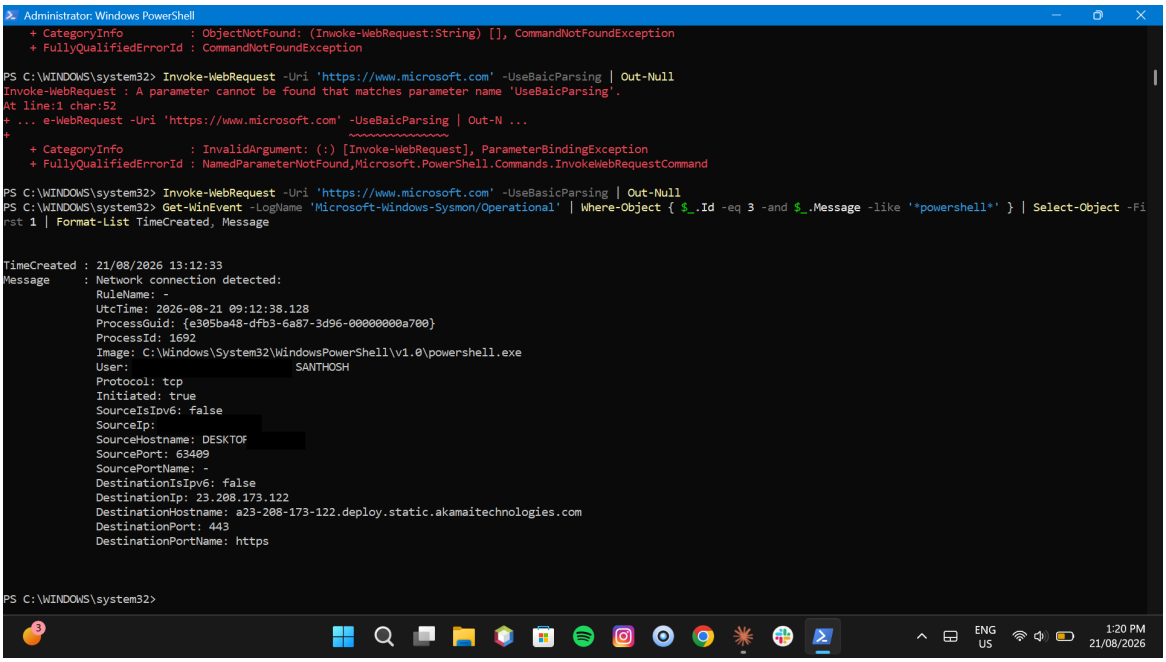


Fig. A10 — Finding 6: Sysmon Event ID 3 (Network Connection) showing powershell.exe connecting outbound to an Akamai CDN edge node (serving microsoft.com) over HTTPS, port 443.

Appendix B: Useful Sysmon Event ID Reference

Event ID	Description

1	Process creation
2	A process changed a file creation time
3	Network connection
5	Process terminated
6	Driver loaded
7	Image loaded
8	CreateRemoteThread
10	ProcessAccess
11	FileCreate
12/13/14	Registry create/modify, set, rename
15	FileCreateStreamHash
17/18	Pipe created / connected
22	DNS query
23	FileDelete